Департамент
образования и науки
города Москвы

Государственное бюджетное
профессиональное образовательное
учреждение города Москвы

Колледж информационных
технологий «ИТ.МОСКВА»

IT.МОСКВА

8 499 612-03-25
spo-mgkeit@edu.mos.ru

115446, г. Москва,
ул. Академика Миллионщикова, д. 20

00216071 ОКПО
1027739491790 ОГРН
7724035635 ИНН
772401001 КПП

# Практическая работа №3. Построение архитектуры программного средства

**Специальность** **09.02.07 Информационные системы и Программирование**

(код, наименование)

**Вид практики** Учебная практика УП.02

**Сроки прохождения практики** 29.09.26 - 12.10.26

**Место прохождения практики** колледж ИТ.МОСКВА

*(название предприятия)*

**Выполнил(-а) студент(-ка)** группы **3-ИП-7-24**

**Муталибова Алина Руслановна**

*(Ф.И.О. студента)*

Москва, 2026

# 1. ПОСТАНОВКА ЗАДАЧИ

**Предметная область:** страховая медицинская компания.

Страховая медицинская компания занимается оформлением полисов медицинского страхования, ведением учёта застрахованных граждан и взаимодействием с медицинскими организациями. Сотрудники компании должны иметь возможность быстро находить сведения о клиентах, проверять срок действия полисов и регистрировать обращения.

При ручной обработке информации могут возникать ошибки, дублирование записей и затруднения при поиске необходимых данных. Поэтому необходимо разработать программное средство, которое позволит хранить информацию в единой базе данных и автоматизировать основные рабочие процессы.

**Цель разработки** — создание информационной системы для учёта клиентов, оформления и проверки полисов, регистрации обращений и формирования отчётов.

Основные задачи разработки:

- определить требования к программному продукту;
- разработать структуру системы;
- описать основные процессы обработки информации;
- определить сущности базы данных и связи между ними;
- разработать схемы взаимодействия компонентов программы;
- описать работу отдельного программного модуля.

# 2. АНАЛИЗ ФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ

Программный продукт должен обеспечивать выполнение следующих функций:

- добавление, редактирование и просмотр сведений о клиентах;
- оформление и изменение медицинских полисов;
- проверка статуса и срока действия полиса;
- хранение сведений о медицинских организациях;
- регистрация и обработка обращений;
- поиск клиентов, полисов и обращений;
- формирование отчётов за выбранный период;
- управление учётными записями сотрудников и правами доступа;
- хранение истории изменений;
- резервное копирование и восстановление данных.

Основными пользователями системы являются администратор, страховой специалист и специалист по обработке обращений.

Администратор управляет пользователями и настройками. Страховой специалист работает с клиентами и полисами. Специалист по обработке обращений регистрирует обращения и изменяет их статусы.

# 3. АНАЛИЗ ЭКСПЛУАТАЦИОННЫХ ТРЕБОВАНИЙ

**Надёжность.** Система должна корректно обрабатывать введённые данные, сообщать об ошибках и обеспечивать возможность восстановления информации после сбоя.

**Условия эксплуатации.** Программа предназначена для использования сотрудниками страховой медицинской компании на рабочих компьютерах с доступом к локальной сети или интернету.

**Технические средства:**

- сервер — процессор от 4 ядер, 16 ГБ оперативной памяти и SSD от 500 ГБ;
- рабочие станции — процессор от 2 ядер, 8 ГБ оперативной памяти и SSD от 256 ГБ;
- сетевое оборудование и многофункциональное устройство для печати документов.

**Программная совместимость:**

- Windows 10/11;
- Linux Ubuntu Server;
- Python 3.11;
- Django;
- PostgreSQL 15;
- Nginx или Apache.

**Безопасность.** Доступ к информации должен предоставляться в соответствии с ролью сотрудника. Пароли необходимо хранить в защищённом виде, а доступ к персональным данным ограничивать.

**Обслуживание.** Необходимо предусмотреть резервное копирование базы данных, обновление программного обеспечения и восстановление данных при необходимости.

# 4. ЭСКИЗНЫЙ ПРОЕКТ

## 4.1. Выбор метода решения и языка программирования

Для разработки программного средства предлагается использовать структурный подход к проектированию и объектно-ориентированный подход при организации программного кода.

В качестве основных технологий выбраны:

- **Python 3.11** — язык программирования;
- **Django** — фреймворк для создания веб-приложения;
- **PostgreSQL 15** — система управления базами данных;
- **MS Visio или draw.io** — средства создания схем;
- **MS Office** — оформление документации.

Python позволяет реализовать обработку данных и основные функции системы. Django предоставляет инструменты для работы с формами, пользователями и базой данных. PostgreSQL используется для хранения связанных между собой сведений о клиентах, полисах и обращениях.

## 4.2. Структура программного продукта

Программный продукт состоит из следующих подсистем:

1. Подсистема авторизации и управления доступом.
2. Подсистема учёта клиентов.
3. Подсистема оформления и проверки полисов.
4. Подсистема учёта медицинских организаций.
5. Подсистема регистрации и обработки обращений.
6. Подсистема формирования отчётов.
7. Подсистема администрирования и резервного копирования.

Каждая подсистема отвечает за определённую часть работы системы. Такое разделение упрощает разработку и дальнейшее изменение программы.

### 4.3. Состав функций программного продукта

**F1** — регистрация, редактирование и поиск клиентов.

**F2** — оформление и просмотр полисов.

**F3** — проверка срока действия и статуса полиса.

**F4** — ведение справочника медицинских организаций.

**F5** — регистрация обращений и изменение их статусов.

**F6** — поиск и фильтрация информации.

**F7** — формирование отчётов.

**F8** — управление пользователями и правами доступа.

**F9** — резервное копирование и восстановление данных.

### 4.4. Режимы функционирования

В системе предусмотрены следующие режимы:

- **Основной режим** — регистрация клиентов, работа с полисами и обращениями.
- **Режим отчётности** — формирование и просмотр отчётов.
- **Режим администрирования** — управление пользователями и настройками.
- **Режим обслуживания** — резервное копирование, восстановление и обновление системы.

# 5. СПЕЦИФИКАЦИИ ПРОЦЕССОВ

## 5.1. Процесс «Регистрация клиента»

**Входные данные:** ФИО, дата рождения, номер телефона и другие необходимые сведения.

**Выходные данные:** запись о клиенте в базе данных и сообщение о результате операции.

**Алгоритм:**

1. Сотрудник вводит данные клиента.
2. Система проверяет заполнение обязательных полей.
3. Проверяется наличие похожей записи.
4. Если обнаружен возможный дубликат, система выводит предупреждение.
5. Если данные корректны, запись сохраняется.
6. Система сообщает об успешной регистрации.

## 5.2. Процесс «Оформление медицинского полиса»

**Входные данные:** идентификатор клиента, номер полиса, дата оформления и дата окончания действия.

**Выходные данные:** запись о полисе и сообщение о результате оформления.

**Алгоритм:**

1. Сотрудник выбирает клиента.
2. Система проверяет наличие клиента в базе данных.
3. Проверяется заполнение обязательных полей.
4. Проверяется уникальность номера полиса.

5. Информация о полисе сохраняется.
6. Система определяет статус полиса с учётом срока действия.
7. Сотрудник получает результат операции.

### 5.3. Процесс «Регистрация страхового случая»

**Входные данные:** номер полиса, сведения о медицинской организации, дата и описание обращения.

**Выходные данные:** запись об обращении и его текущий статус.

**Алгоритм:**

1. Сотрудник указывает номер полиса.
2. Система находит полис и проверяет его статус.
3. Проверяется наличие необходимых данных.
4. Создаётся запись об обращении.
5. Обращению присваивается начальный статус.
6. Данные сохраняются в базе.
7. Система выводит сообщение о результате.

### 5.4. Процесс «Формирование отчёта»

**Входные данные:** тип отчёта и выбранный период.

**Выходные данные:** отчёт с информацией о клиентах, полисах или обращениях.

**Алгоритм:**

1. Сотрудник выбирает тип отчёта.
2. Указывает период.
3. Система получает соответствующие данные из базы.
4. Информация обрабатывается и оформляется в виде отчёта.

5. Отчёт отображается пользователю или сохраняется в файл.

# 6. ДИАГРАММЫ ПОТОКОВ ДАННЫХ (DFD)

Диаграммы потоков данных показывают, какая информация поступает в систему, как она обрабатывается и где хранится.

## 6.1. Контекстная диаграмма (уровень 0)

**Основной процесс:** информационная система страховой медицинской компании.

**Внешние сущности:**

- клиент;
- страховой специалист;
- медицинская организация;
- руководство компании.

**Схема:**




## 6.2. Диаграмма потоков данных (1 уровень)

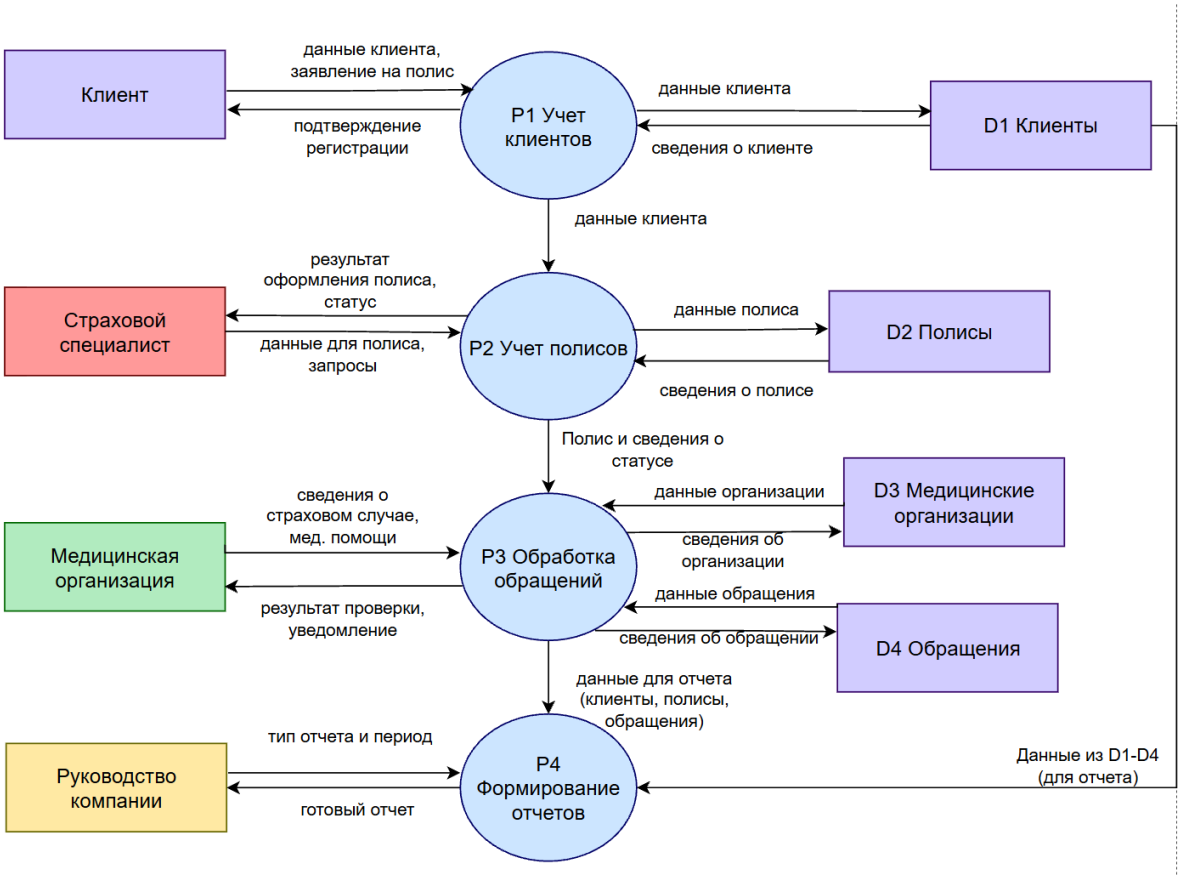
На первом уровне основной процесс разделяется на четыре подпроцесса:

- **P1 — Учёт клиентов**;

– **P2 — Учёт полисов**;
– **P3 — Обработка обращений**;
– **P4 — Формирование отчетов**.

Хранилища данных:

– **D1 — Клиенты**;
– **D2 — Полисы**;
– **D3 — Медицинские организации**;
– **D4 — Обращения**;

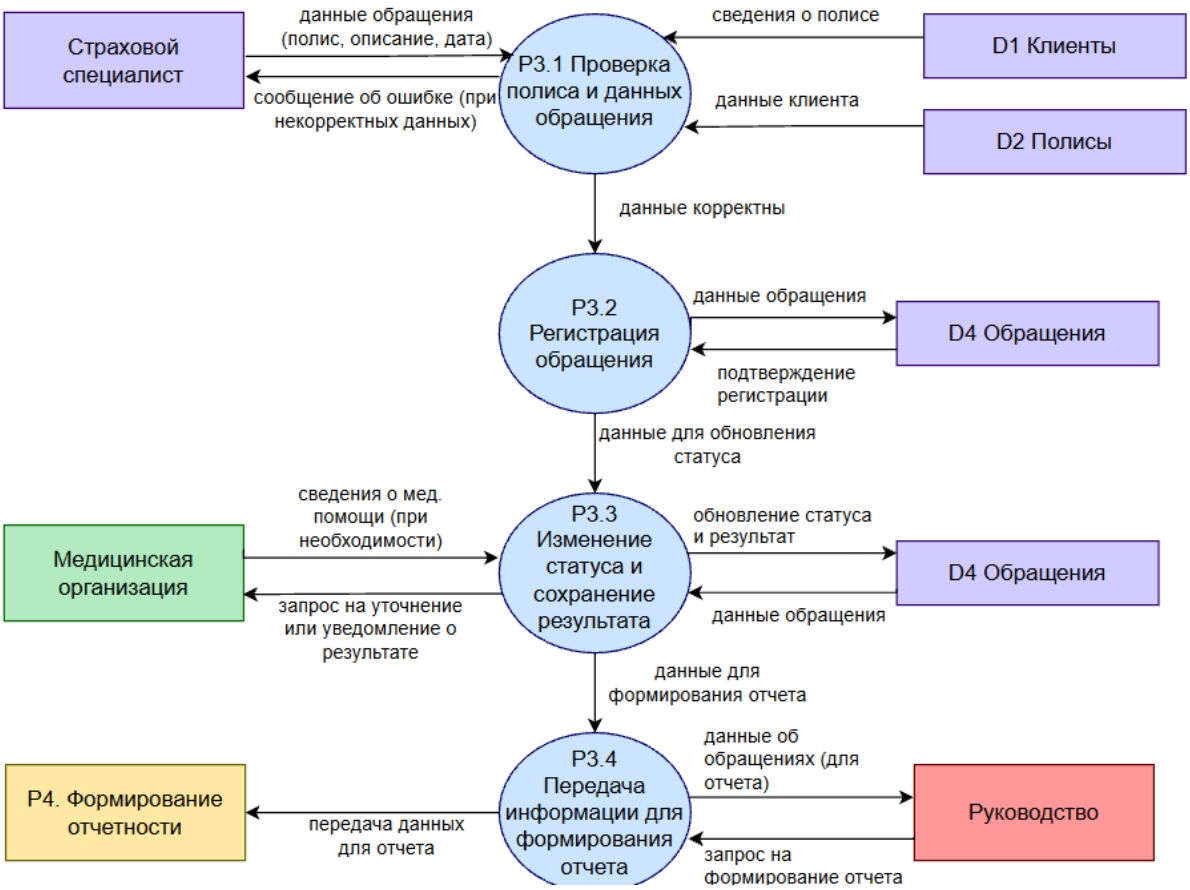



### 6.3. Диаграмма обработки обращения (уровня 2)

Процесс P3 разделяется на следующие этапы:

– **P3.1** — проверка полиса и данных обращения;
– **P3.2** — регистрация обращения;

– **Р3.3** — изменение статуса и сохранение результата;
– **Р3.4** — передача информации для формирования отчёта.




Если данные корректны, система регистрирует обращение. Если обнаружена ошибка, сотрудник получает сообщение и может исправить информацию.

# 7. ДИАГРАММА «СУЩНОСТЬ-СВЯЗЬ» (ERD)

Для хранения информации используются следующие основные сущности:

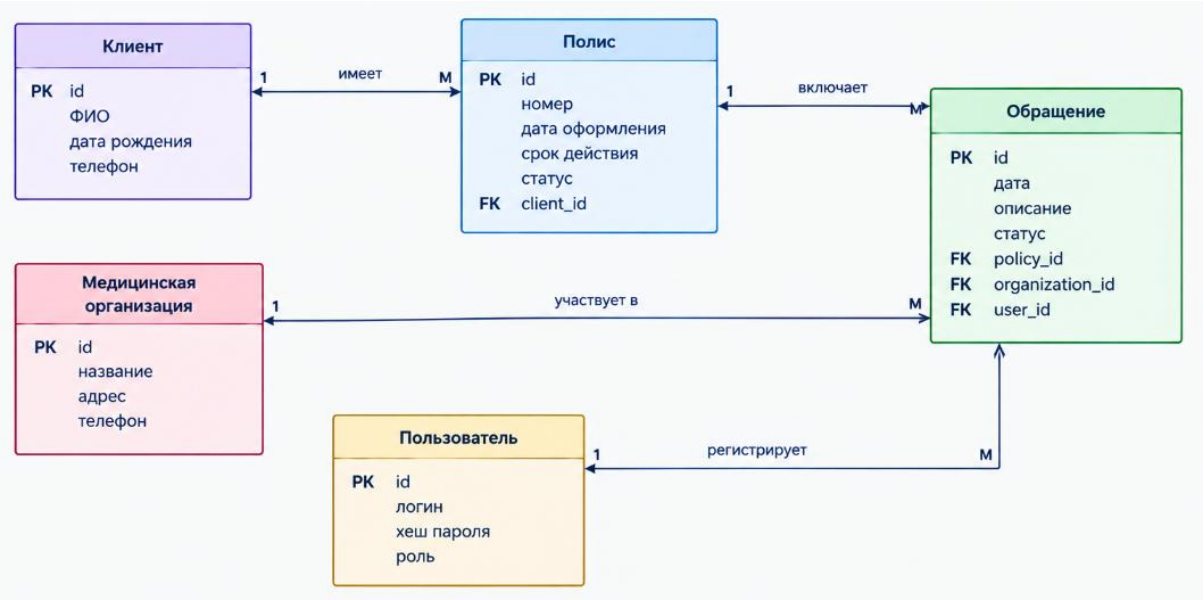
1. Клиент.
2. Полис.
3. Медицинская организация.
4. Обращение.
5. Пользователь.

| Сущность | Основные атрибуты |
|---|---|
| Клиент | id, ФИО, дата рождения, телефон |
| Полис | id, номер, дата оформления, срок действия, статус, client_id |
| Медицинская организация | id, название, адрес, телефон |
| Обращение | id, дата, описание, статус, policy_id, organization_id, user_id |
| Пользователь | id, логин, хеш пароля, роль |

### 7.2. Связи между сущностями

- **Клиент (1) — (M) Полис:** у одного клиента может быть несколько полисов.
- **Полис (1) — (M) Обращение:** к одному полису может относиться несколько обращений.
- **Медицинская организация (1) — (M) Обращение:** одна организация может участвовать в обработке нескольких обращений.
- **Пользователь (1) — (M) Обращение:** один сотрудник может зарегистрировать несколько обращений.

Обозначение **1** означает связь «один ко многим».




Такая структура позволяет хранить информацию в связанных таблицах и находить необходимые сведения без повторного ввода всех данных.

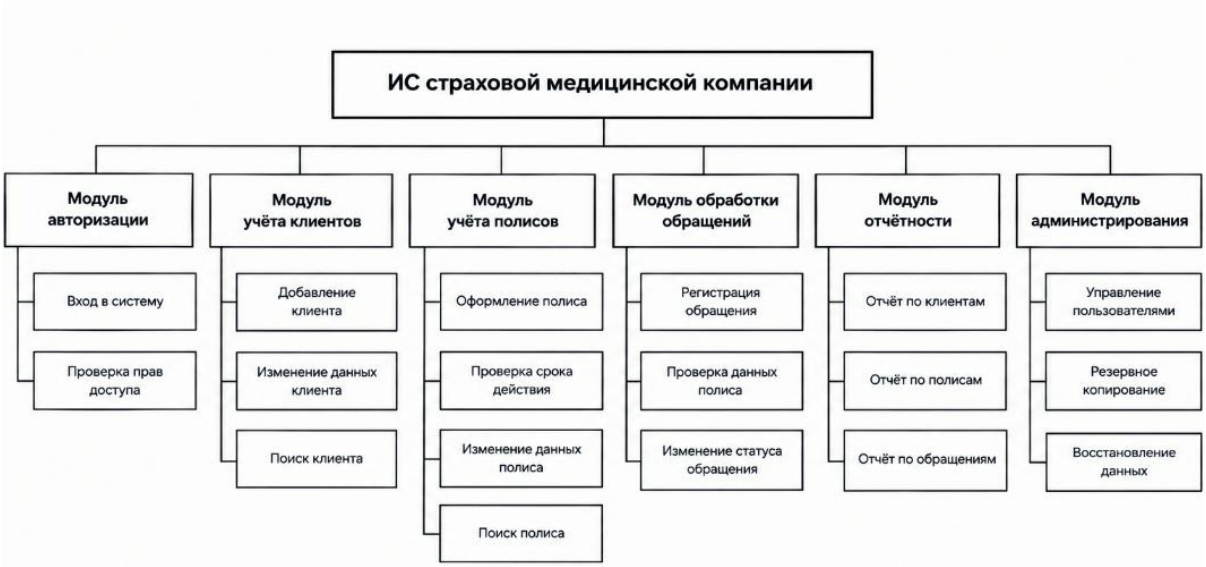
# 8. СЛОВАРЬ ТЕРМИНОВ

| Термин | Определение |
|---|---|
| **Клиент** | Человек, зарегистрированный в системе страховой медицинской компании |
| **Медицинский полис** | Документ или запись, подтверждающая наличие медицинского страхования |
| **Обращение** | Зарегистрированное событие или запрос, связанный с использованием полиса |
| **Медицинская организация** | Медицинское учреждение, сведения о котором хранятся в системе |
| **Страховой специалист** | Сотрудник, который регистрирует клиентов и оформляет полисы |
| **Администратор** | Сотрудник, управляющий пользователями и настройками системы |
| **DFD** | Диаграмма потоков данных, показывающая движение информации в системе |
| **ERD** | Диаграмма «сущность-связь», описывающая данные и отношения между нимм |
| **СУБД** | Система управления базами данных |

| | |
|---|---|
| **Подсистема** | Часть программного продукта, выполняющая определённые функции |
| **Страховой случай** | событие, при котором клиент имеет право на получение медицинской помощи по полису |
| **Направление** | документ, по которому клиент получает лечение в медицинской организации |
| **Статус обращения** | текущее состояние обработки обращения (например, «в обработке», «закрыто») |

# 9. ТЕХНИЧЕСКИЙ ПРОЕКТ

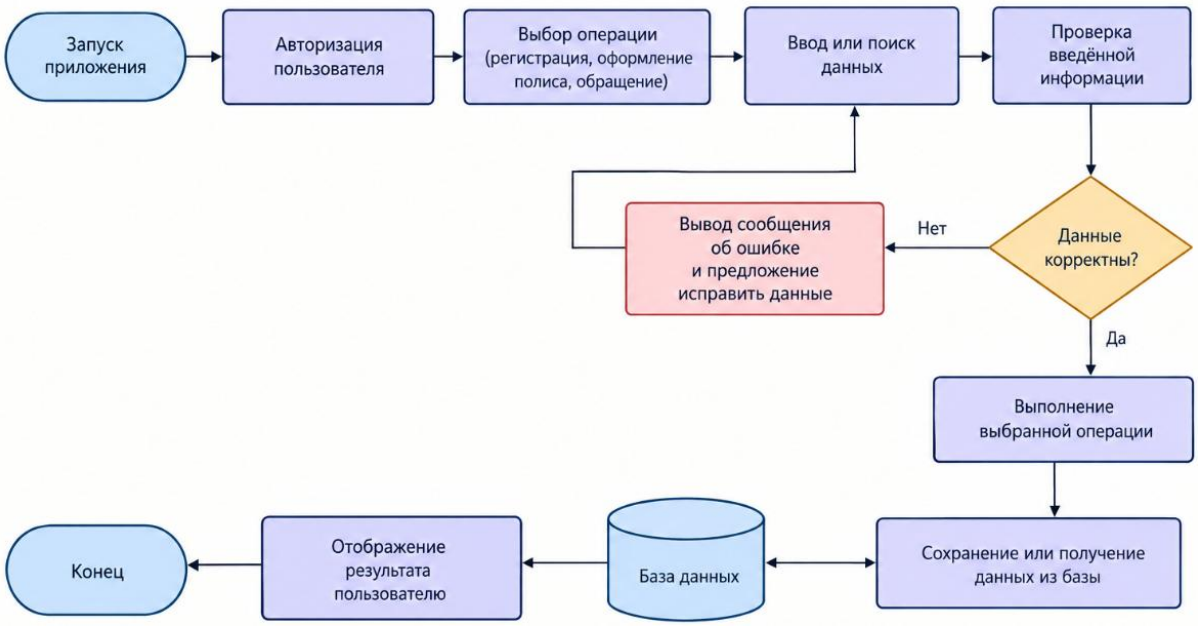
Структурная схема показывает состав информационной системы страховой медицинской компании и её разбиение на функциональные модули.




Каждый модуль содержит набор функций и взаимодействует с базой данных, в которой хранятся сведения о клиентах, полисах, медицинских организациях, обращениях и пользователях.

# 10. ФУНКЦИОНАЛЬНАЯ СХЕМА

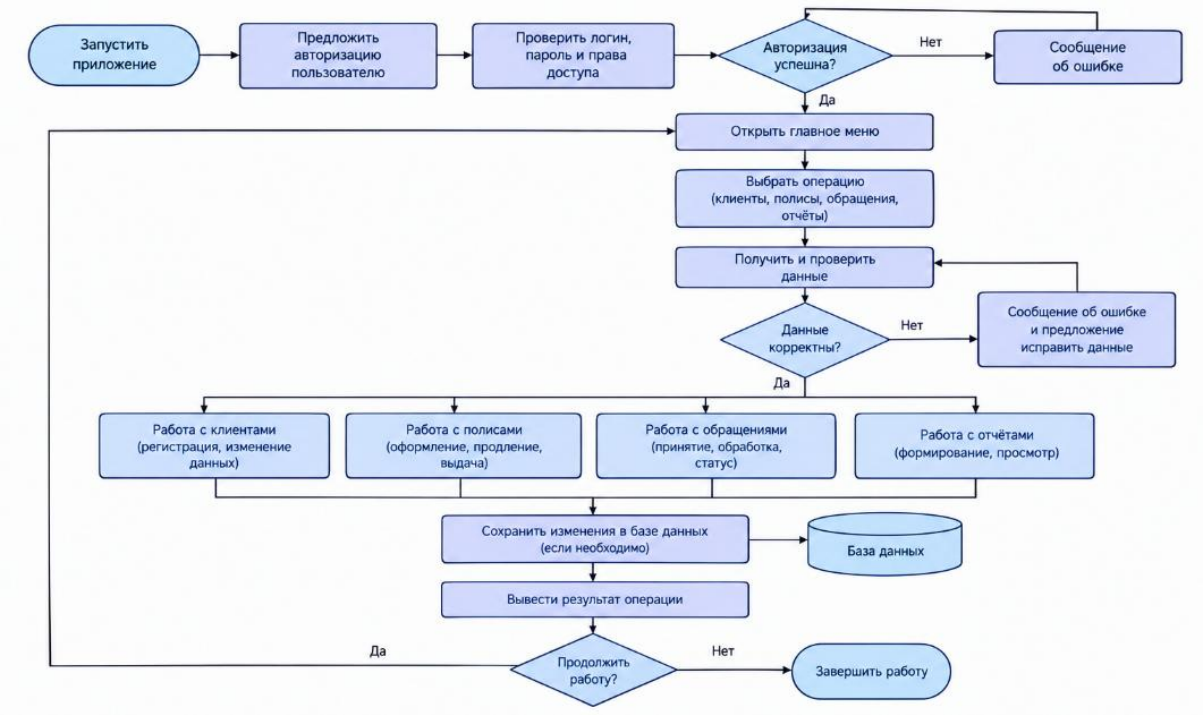
Функциональная схема показывает последовательность действий пользователя и системы при выполнении основной операции.




Данная схема описывает общий порядок работы системы при регистрации клиента, оформлении полиса и обработке обращения.

# 11. АЛГОРИТМ ПРОГРАММЫ

Общий алгоритм работы программного продукта:




Если при обработке данных возникает ошибка, система должна вывести понятное сообщение и не сохранять некорректную запись.

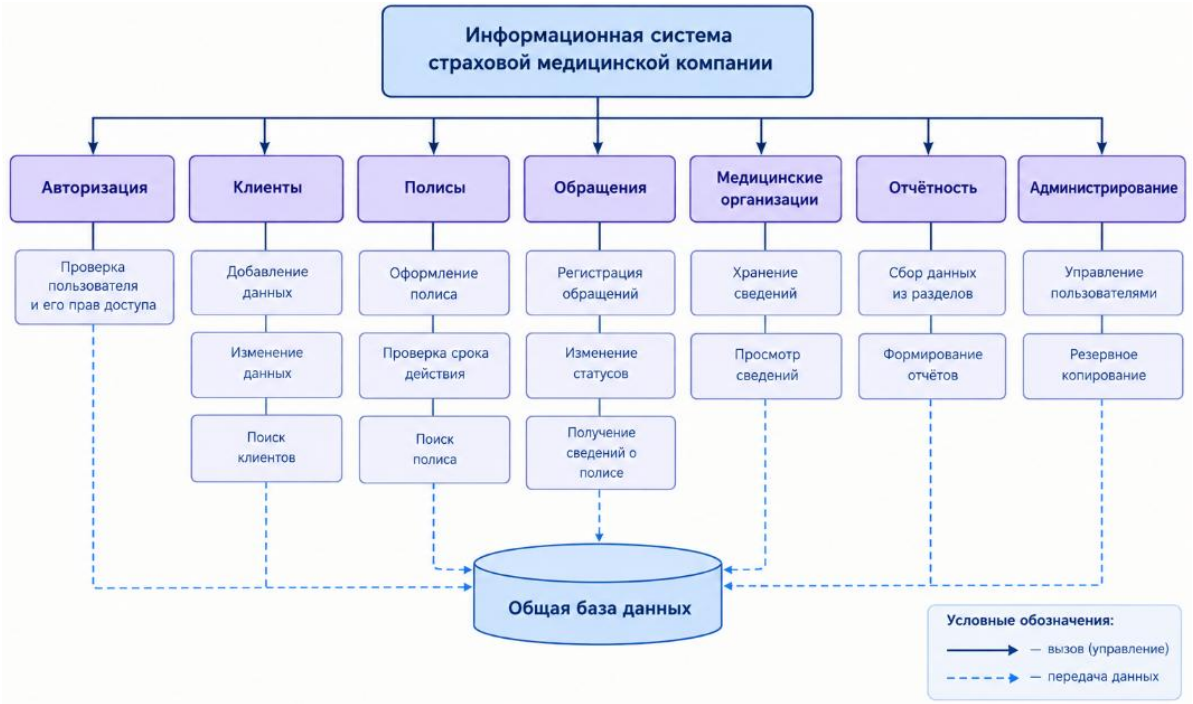
# 12. СТРУКТУРНАЯ КАРТА КОНСТАНТАЙНА

Структурная карта Константайна показывает разделение программного продукта на модули и взаимодействие между ними.

**Главный модуль: «Информационная система страховой медицинской компании»**

Он включает следующие модули:

- **Авторизация** — проверка пользователя и его прав доступа.
- **Клиенты** — добавление, изменение и поиск данных.
- **Полисы** — оформление и проверка срока действия.
- **Обращения** — регистрация обращений и изменение их статусов.
- **Медицинские организации** — хранение и просмотр сведений.
- **Отчётность** — формирование отчётов.
- **Администрирование** — управление пользователями и резервным копированием.




Модули взаимодействуют через главный программный компонент и используют общую базу данных. Например, модуль обращений получает сведения о полисе, а модуль отчётности использует информацию из нескольких разделов системы.

# 13. СТРУКТУРНАЯ КАРТА ДЖЕКСОНА

Структурная карта Джексона описывает структуру программы на основе обрабатываемых данных и последовательности операций.

Для страховой медицинской компании обработка информации организована следующим образом:




Таким образом, программа последовательно получает данные, проверяет их, выполняет необходимые действия и возвращает результат.

# 14. ТЕХНИЧЕСКИЙ ПРОЕКТ ПРОГРАММНОГО МОДУЛЯ

## 14.1. Модуль «Оформление и проверка медицинского полиса»

**Назначение модуля:** оформление нового медицинского полиса и проверка срока его действия.

**Входные данные:**

- идентификатор клиента;
- номер полиса;
- дата оформления;
- дата окончания действия.

**Выходные данные:**

- сохранённая запись о полисе;
- результат проверки;
- сообщение пользователю.

## 14.2. Основные этапы работы модуля

1. Получить данные клиента и полиса.
2. Проверить наличие клиента в базе данных.
3. Проверить заполнение обязательных полей.
4. Проверить уникальность номера полиса.
5. Проверить корректность дат.
6. Сохранить данные, если все проверки пройдены.
7. Определить статус полиса с учётом срока действия.
8. Вывести результат пользователю.

### 14.3. Требования к модулю

Модуль должен:

- проверять корректность введённых данных;
- не допускать повторения номера полиса;
- проверять наличие клиента в базе;
- корректно обрабатывать ошибки;
- ограничивать доступ к операциям в соответствии с ролью пользователя;
- сохранять изменения в базе данных.

Данный модуль является одной из основных частей системы, поскольку позволяет сотрудникам оформлять полисы и проверять их актуальность.

# 15. ИТОГ

В ходе практической работы была рассмотрена предметная область страховой медицинской компании и определены основные требования к программному продукту.

Были описаны функции системы, условия её эксплуатации и основные технические решения. Разработаны спецификации процессов регистрации клиентов, оформления полисов, обработки обращений и формирования отчётов.

Также определены основные сущности базы данных и связи между ними, рассмотрены диаграммы потоков данных, структурная и функциональная схемы, алгоритм работы программы и структурные карты Константайна и Джексона.

В завершение был описан технический проект модуля оформления и проверки медицинского полиса.